HEALTH AI

Co-Creation & Innovation Platform

Software Design Document (SDD)

Project Name	HEALTH AI Co-Creation & Innovation Platform
Document Type	Software Design Document (SDD)
Version	1.0
Date	April 2026
Project	Erasmus+ HEALTH AI Project
Repository	github.com/goksubulut/health-ai-cocreation
Tech Stack	React 18 + Vite | Node.js 20 + Express 4 | MySQL 8 | Docker

Erasmus+ HEALTH AI Project • April 2026

Table of Contents

Table of Contents....2
Revision History....4
1. Introduction....5
1.1 Purpose....5
1.2 Scope....5
1.3 Reference Documents....5
1.4 Definitions and Abbreviations....5
2. Architectural Design....7
2.1 Architectural Overview....7
2.2 Layers....7
2.3 Deployment Diagram....7
3. Component Design....9
3.1 Component List....9
3.2 Component Interaction....10
4. Database Design....11
4.1 ER Diagram....11
4.2 Table Definitions....11
4.2.1 users....11
4.2.2 posts....12
4.2.3 meeting_requests....13
4.2.4 time_slots....14
4.2.5 nda_acceptances....14
4.2.6 activity_logs....15
4.2.7 meeting_feedbacks (DB Table Exists — Backend Not Yet Implemented)....15
5. API Design....17
5.1 Authentication — /api/auth....17
5.2 Post Management — /api/posts....17
5.3 Meeting Requests — /api/meetings....18
5.3.1 Meeting Feedback — /api/meetings/:id/feedback (Planned)....19
5.4 Admin — /api/admin....19
5.5 User — /api/users....20
6. UI Design....21
6.1 Screen List and Navigation....21
6.2 Frontend State Management....22
6.3 Wireframes — Key Screens....22

7. Security Design 24
7.1 Error Handling Strategy 25
7.2 Migration Strategy 26
8. State Diagrams 27
8.1 Post Lifecycle 27
8.2 Meeting Request Lifecycle 27
9. SRS–SDD Traceability Matrix 29
10. Appendices 31
10.1 Project File Structure 31
10.2 Environment Variables Reference 31
10.3 Match Score Algorithm 32

Revision History

Date	Version	Description	Author
April 2026	1.0	Initial release	Development Team

1. Introduction

1.1 Purpose

This Software Design Document describes the architectural, component-level, database, and API design of the HEALTH AI Co-Creation & Innovation Platform. It translates the functional and non-functional requirements defined in the Software Requirements Specification (SRS v1.0) into concrete technical design decisions that guide implementation.

The document is intended for the development team, project reviewers, and future maintainers of the system. It covers all major system layers — from the React frontend through the Express API layer to the MySQL database — and includes security, GDPR, and deployment design.

1.2 Scope

The design covers the following system capabilities as defined in the SRS:

- Secure user registration and authentication with institutional (.edu / .edu.tr) email enforcement
- Role-based collaboration: healthcare professionals post projects; engineers discover and respond
- Post lifecycle management: creation, publication, expiry, and partner confirmation
- Meeting request workflow with NDA acceptance, time slot negotiation, and scheduling
- City-based and expertise-based partner matching with a weighted score algorithm
- Admin panel: user management, post moderation, statistics, and audit log management
- GDPR compliance: account anonymisation, personal data export, IP hashing
- Containerised deployment with Docker Compose (MySQL + backend + frontend)

1.3 Reference Documents

- HEALTH AI SRS v1.0 — Software Requirements Specification (Filled), April 2026
- HEALTH AI Backend Architecture Plan (HEALTH_AI_Backend_Architecture.md), April 2026
- Erasmus+ HEALTH AI Project Brief (co-creation_platform_project_brief-revised.docx)
- Source code repository: github.com/goksubulut/health-ai-cocreation

1.4 Definitions and Abbreviations

- **API:** Application Programming Interface
- **REST:** Representational State Transfer
- **JWT:** JSON Web Token — stateless authentication mechanism
- **ORM:** Object-Relational Mapper — Sequelize 6 used in this project
- **RBAC:** Role-Based Access Control
- **GDPR:** General Data Protection Regulation (EU 2016/679)
- **NDA / MNDA:** Non-Disclosure Agreement / Mutual Non-Disclosure Agreement
- **SPA:** Single Page Application (React frontend)
- **SMTP:** Simple Mail Transfer Protocol — used by Nodemailer for email delivery
- **DXA:** Document Exchange Architecture unit (1/20 of a point, used in DOCX layout)

- **CSP:** Content Security Policy
- **CORS:** Cross-Origin Resource Sharing
- **TTL:** Time-To-Live — used for token expiry durations
- **SRS:** Software Requirements Specification
- **SDD:** Software Design Document (this document)

2. Architectural Design

2.1 Architectural Overview

The HEALTH AI platform follows a classic three-tier Client-Server architecture with clear separation of concerns across the Presentation, Business Logic, Data Access, and Database layers. The system is deployed as a monorepo with a React SPA frontend served through the same Express server as the REST API.

> *Architectural pattern: Three-Tier Layered Architecture (Presentation → Business Logic → Data Access → Database) with a RESTful API contract between tiers.*

In development mode, the frontend Vite dev server proxies /api and /health requests to the Express backend (port 3001). In production and Docker deployments, Express serves the compiled frontend dist alongside the API from a single server, eliminating CORS complexity.

The backend itself is structured as a Model-View-Controller (MVC) pattern within the Express application: routes act as the Router (URL-to-controller mapping), controllers implement the Business Logic, Sequelize models form the Data Access layer, and JSON responses constitute the View (data presented to the client). In a REST API context there is no HTML template engine — the View is the serialised JSON payload returned by each controller action.

2.2 Layers

Layer	Responsibility	Technology
Presentation	User interface, page rendering, client-side routing	React 18, Vite, TypeScript, Tailwind CSS
Business Logic	Request handling, workflow orchestration, validation	Node.js 20, Express 4, Joi
Data Access	ORM model definitions, associations, migrations	Sequelize 6 ORM
Database	Persistent relational data storage	MySQL 8

All four layers communicate strictly through defined interfaces: the Presentation layer consumes REST API endpoints (JSON over HTTP), the Business Logic layer queries the Data Access layer through Sequelize model methods, and the Data Access layer communicates exclusively with MySQL through Sequelize's connection pool.

2.3 Deployment Diagram

The system is containerised using Docker Compose, which orchestrates two services: the MySQL 8 database and the Node.js application server. The application server compiles the React frontend at build time and serves both the API and the static assets.

> *Deployment: Docker Compose → healthai-app (Node.js 20, port 3001) + healthai-db (MySQL 8.4, port 3306). Frontend compiled into dist/ at build time and served as static files by Express.*

Component layout in the Docker Compose deployment:

- healthai-db: MySQL 8.4 container. Persistent volume (db_data) for data durability. Health check on mysqladmin ping before app starts.

- healthai-app: Built from Dockerfile. Depends on healthai-db (condition: service_healthy). Serves API at /api/* and compiled React SPA from dist/. Exposed on port 3001.
- Environment configuration: all secrets injected via environment variables (JWT secrets, SMTP credentials, DB password). No hardcoded secrets in source.
- HTTPS termination: handled by an upstream reverse proxy (e.g. nginx, Traefik, or cloud load balancer) in production deployments.

3. Component Design

3.1 Component List

The system is decomposed into the following logical modules, each with a clearly scoped responsibility:

Component	Responsibility	Technology / Files
AuthModule	Registration, login, email verification, JWT token management, password reset	`authController.js`, `authSchemas.js`, `emailService.js`
PostModule	Post CRUD, lifecycle state machine, post expiry, confidentiality control	`postController.js`, `postSchemas.js`
MatchingModule	City-based post matching, expertise scoring, post filtering and search	`postController.js (getMatches)`, `matchScore.js`
MeetingModule	Meeting requests, NDA acceptance, time slot proposals, slot confirmation	`meetingController.js`, `meetingSchemas.js`
UserModule	Profile management, password change, GDPR data export, account deletion	`userController.js`, `userSchemas.js`
AdminModule	User/post management, platform statistics, dashboard	`adminController.js`, `admin.js (routes)`
AuditModule	Activity logging, log querying and filtering, CSV export, log cleanup	`logService.js`, `ActivityLog.js`
MeetingFeedbackModule (Planned)	Post-meeting partner ratings. After a meeting is scheduled, each party can submit trust_rating (1-5), experience_rating (1-5) and notes. UNIQUE(meeting_request_id, user_id) prevents duplicate submissions. DB table exists; Sequelize model, controller and API routes not yet implemented.	`MeetingFeedback.js`, `feedbackController.js (planned)`
PlatformFeedbackModule (UI Only)	General product feedback widget available on all authenticated pages. Collects emoji rating (very-sad/sad/neutral/happy), free-text message, and optional email. Currently frontend-only — onSubmit handler logs to console. No backend endpoint or persistence layer exists yet.	`feedback-widget.tsx`, `Layout.jsx`
NotificationModule	Transactional email notifications. Trigger to recipient: (1) Register: user gets verification link. (2) Password reset: user gets reset link. (3)	`emailService.js (Nodemailer/SMTP)`

	Meeting request created: post owner notified. (4) Meeting accepted: requester notified. (5) Meeting declined: requester notified. (6) Slot proposed: other party notified. (7) Slot confirmed / scheduled: both parties notified. (8) Meeting cancelled: other party notified. All sends are fire-and-forget; failures never block the API response.	

3.2 Component Interaction

The following describes how the main components communicate during the platform's core workflows:

Authentication flow: Client → AuthModule (register/login) → NotificationModule (verification email) → AuditModule (LOGIN/REGISTER log). On subsequent requests: Client sends Bearer token → authenticate.js middleware → authorize.js middleware → target controller.

Post creation flow: PostModule.createPost() → Post.create() (Sequelize) → AuditModule.logActivity(POST_CREATE). Post discovery calls expireStalePosts() on every GET to enforce time-based expiry without a background cron job.

Meeting request flow: MeetingModule.createMeetingRequest() validates NDA, creates MeetingRequest record → NdaAcceptance.findOrCreate() → NotificationModule.sendMeetingRequestNotification(). Accept/decline/slot-confirm each update MeetingRequest status, cascade-update Post status, and trigger email notifications.

Matching flow: Frontend matchScore.js calculates a weighted score (expertise 50 pts, city 30 pts, prototype stage 10 pts, freshness <3 days 10 pts) client-side. Server-side getMatches() endpoint filters by city equality for the dashboard discovery feed.

Admin flow: All /api/admin/* routes require authenticate + authorize('admin') middleware chain. AdminModule reads directly from User, Post, MeetingRequest, and ActivityLog models. Log cleanup and CSV export use raw Sequelize queries for performance.

4. Database Design

4.1 ER Diagram

> *Entity Relationships: users ←1:N→ posts | posts ←1:N→ meeting_requests | users ←1:N→ meeting_requests (requester + post_owner) | meeting_requests ←1:N→ time_slots | users + posts ←N:M→ nda_acceptances | users ←1:N→ activity_logs*

All foreign key relationships are enforced at both the database level (MySQL foreign key constraints via Sequelize migrations) and the application level (Sequelize model associations defined in models/index.js).

Key design decisions:

- Sequelize underscored: true maps camelCase model fields to snake_case database columns automatically throughout the system.
- The MeetingRequest model carries a proposed_time field (a single lightweight datetime suggestion that either party can set directly via PATCH /api/meetings/:id/propose-time) alongside the time_slots table (which supports up to 5 structured slot proposals from the requester). These two mechanisms serve different use cases: proposed_time is a quick informal suggestion exchanged before formal scheduling, while time_slots drives the structured accept-and-confirm workflow. The confirmed_slot field on MeetingRequest is populated only when a time_slots entry is confirmed via the /slots/:slotId/confirm endpoint. To prevent data inconsistency, the system must treat proposed_time and confirmed_slot as mutually exclusive final states — once a slot is confirmed via time_slots, proposed_time should be ignored. A future improvement would be to remove proposed_time and consolidate all scheduling through time_slots.
- nda_acceptances uses a composite UNIQUE constraint on (user_id, post_id) to prevent duplicate NDA records while allowing audit-safe updates via findOrCreate.
- activity_logs uses BIGINT for the id field (large volume expected) and indexes on user_id, action_type, and created_at for efficient admin-panel queries.

4.2 Table Definitions

4.2.1 users

Field	Type	Req.	Constraint	Description
`id`	INT	Yes	PK, AUTO_INCREMENT	Unique user identifier
`email`	VARCHAR(255)	Yes	UNIQUE, .edu/.edu.tr check	Institutional email address
`password_hash`	VARCHAR(255)	Yes	—	bcrypt hash (saltRounds: 12)
`role`	ENUM	Yes	engineer | healthcare | admin	User role for RBAC
`first_name`	VARCHAR(100)	Yes	—	User's first name
`last_name`	VARCHAR(100)	Yes	—	User's last name
`institution`	VARCHAR(255)	No	—	University/hospital affiliation

city	VARCHAR(100)	No	—	Used for city-based matching
country	VARCHAR(100)	No	—	User's country
expertise	VARCHAR(255)	No	—	Professional expertise keywords
is_verified	BOOLEAN	Yes	DEFAULT FALSE	Email verification status
is_active	BOOLEAN	Yes	DEFAULT TRUE	Account active/suspended flag
verify_token	VARCHAR(255)	No	Nullable	Email verification token (6h TTL)
verify_expiry	DATETIME	No	Nullable	Verification token expiry
reset_token	VARCHAR(255)	No	Nullable	Password reset token (1h TTL)
reset_expiry	DATETIME	No	Nullable	Password reset token expiry
last_login	DATETIME	No	Nullable	Timestamp of last successful login
created_at	DATETIME	Yes	DEFAULT NOW()	Record creation timestamp
updated_at	DATETIME	Yes	ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

4.2.2 posts

Field	Type	Req.	Constraint	Description
id	INT	Yes	PK, AUTO_INCREMENT	Unique post identifier
user_id	INT	Yes	FK → users(id)	Post author
title	VARCHAR(255)	Yes	—	Post title
domain	VARCHAR(100)	Yes	—	Medical/engineering domain (e.g. Cardiology, ML)
description	TEXT	Yes	—	Full project description
required_expertise	VARCHAR(255)	No	—	Skills sought from partner
project_stage	ENUM	No	idea | concept_validation | prototype | pilot | pre_deployment	Current project phase

commitment_level	ENUM	No	advisor | co_founder | research_partner	Expected collaboration level
confidentiality	ENUM	Yes	public | meeting_only	Description visibility control
status	ENUM	Yes	draft | active | meeting_scheduled | partner_found | expired | removed_by_admin	Post lifecycle state
city	VARCHAR(100)	No	—	Post location (used for matching)
country	VARCHAR(100)	No	—	Post country
expiry_date	DATE	No	Nullable	Post auto-expiry date
auto_close	BOOLEAN	Yes	DEFAULT FALSE	Auto-close on expiry date
created_at	DATETIME	Yes	DEFAULT NOW()	Creation timestamp
updated_at	DATETIME	Yes	ON UPDATE	Last update timestamp

4.2.3 meeting_requests

Field	Type	Req.	Constraint	Description
id	INT	Yes	PK, AUTO_INCREMENT	Meeting request identifier
post_id	INT	Yes	FK → posts(id)	Related collaboration post
requester_id	INT	Yes	FK → users(id)	User who sent the request
post_owner_id	INT	Yes	FK → users(id) — denormalised for query performance	Redundant copy of posts.user_id. Avoids a JOIN on every meeting lookup. Acceptable trade-off for read-heavy meeting feeds; must be kept in sync at insert time (enforced by controller).
message	TEXT	No	Nullable	Optional introduction message
nda_accepted	BOOLEAN	Yes	DEFAULT FALSE	Whether NDA was accepted at creation
nda_accepted_at	DATETIME	No	Nullable	Timestamp of NDA acceptance
status	ENUM	Yes	pending | accepted | declined | scheduled | cancelled	Request lifecycle state
proposed_time	DATETIME	No	Nullable	Proposed meeting datetime

confirmed_slot	DATETIME	No	Nullable	Confirmed meeting time after slot agreement
created_at	DATETIME	Yes	DEFAULT NOW()	Creation timestamp
updated_at	DATETIME	Yes	ON UPDATE	Last update timestamp

4.2.4 time_slots

Field	Type	Req.	Constraint	Description
id	INT	Yes	PK, AUTO_INCREMENT	Slot identifier
meeting_request_id	INT	Yes	FK → meeting_requests(id)	Parent meeting request
proposed_by	INT	Yes	FK → users(id)	User who proposed this slot
slot_datetime	DATETIME	Yes	—	Proposed meeting date and time
is_selected	BOOLEAN	Yes	DEFAULT FALSE	Whether this slot was confirmed
created_at	DATETIME	Yes	DEFAULT NOW()	Creation timestamp (Sequelize auto)
updated_at	DATETIME	Yes	ON UPDATE CURRENT_TIMESTAMP	Last update timestamp (Sequelize auto; present because timestamps: true default is active)

4.2.5 nda_acceptances

Field	Type	Req.	Constraint	Description
id	INT	Yes	PK, AUTO_INCREMENT	Acceptance record identifier
user_id	INT	Yes	FK → users(id)	User who accepted
post_id	INT	Yes	FK → posts(id), UNIQUE(user_id, post_id)	Post for which NDA was accepted
ip_hash	VARCHAR(64)	No	SHA-256 hash	Hashed client IP address for audit trail (SHA-256, never raw)
accepted_at	DATETIME	Yes	NOT NULL, set at acceptance time	Timestamp of NDA acceptance. Required for audit trail — a record without this value has no legal standing. Set by

				meetingController.js at the moment the user accepts.
created_at	DATETIME	Yes	DEFAULT NOW(), Sequelize auto	Record creation timestamp. updatedAt is disabled (updatedAt: false in model options)

4.2.6 activity_logs

Field	Type	Req.	Constraint	Description
id	BIGINT	Yes	PK, AUTO_INCREMENT	Log entry identifier
user_id	INT	No	FK → users(id), Nullable	User who performed the action
role	ENUM	No	engineer | healthcare | admin	User's role at time of action
action_type	ENUM	Yes	Indexed; values: LOGIN, FAILED_LOGIN, LOGOUT, REGISTER, EMAIL_VERIFIED, PASSWORD_RESET, POST_CREATE, POST_CLOSE, POST_ADMIN_REMOVE, MEETING_REQUEST, MEETING_ACCEPT, MEETING_DECLINE, SLOT_CONFIRMED, MEETING_CANCEL, ACCOUNT_SUSPEND, ACCOUNT_REACTIVATE	Action type as a fixed ENUM (defined as LOGGABLE_ACTIONS array in ActivityLog.js)
target_entity	VARCHAR(100)	No	Nullable	Entity type affected (post, user, etc.)
target_id	INT	No	Nullable	ID of the affected entity
result_status	ENUM	Yes	success | failure	Outcome of the action
ip_hash	VARCHAR(64)	No	SHA-256 hash	Hashed client IP address (GDPR-safe)
created_at	DATETIME	Yes	DEFAULT NOW(), Indexed	Log creation timestamp

4.2.7 meeting_feedbacks (DB Table Exists — Backend Not Yet Implemented)

> *The meeting_feedbacks table is present in the database schema and is referenced in the SRS. As of v1.0, there is no Sequelize model file (MeetingFeedback.js), no controller, and no API route. The table is ready for implementation in a future sprint.*

Field	Type	Req.	Constraint	Description

`id`	INT	Yes	PK, AUTO_INCREMENT	Feedback record identifier
`meeting_request_id`	INT	Yes	FK → meeting_requests(id), UNIQUE(meeting_request_id, user_id)	The scheduled meeting this feedback belongs to
`user_id`	INT	Yes	FK → users(id)	User submitting the feedback (requester or post owner)
`trust_rating`	TINYINT	No	1–5, Nullable	Trust score for the collaboration partner (1 = low, 5 = high)
`experience_rating`	TINYINT	No	1–5, Nullable	Overall meeting experience score (1 = poor, 5 = excellent)
`notes`	TEXT	No	Nullable	Optional free-text feedback or comments
`created_at`	DATETIME	Yes	DEFAULT NOW(), Sequelize auto	Feedback submission timestamp
`updated_at`	DATETIME	Yes	ON UPDATE CURRENT_TIMESTAMP, Sequelize auto	Last update timestamp

5. API Design

All endpoints return JSON. Protected endpoints require an Authorization: Bearer <access_token> header. Error responses follow the format { "message": "...", "error"?: "..." } with appropriate HTTP status codes. The base URL in development is http://localhost:3001/api and in production it is served from the same origin as the frontend.

Paginated list endpoints (GET /api/posts, /api/posts/mine, /api/admin/users, /api/admin/posts, /api/admin/logs) return a consistent envelope:

> *{ "total": <int>, "page": <int>, "limit": <int>, "totalPages": <int>, "data": [...] }*

Default page size is 10 for public endpoints and 20 for admin endpoints. Maximum limit is capped server-side to prevent excessive queries.

Paginated list endpoints (GET /api/posts, /api/posts/mine, /api/admin/users, /api/admin/posts, /api/admin/logs) return a consistent envelope:

> *{ "total": <int>, "page": <int>, "limit": <int>, "totalPages": <int>, "data": [...] } — for list endpoints. { "data": { ... } } — for single-resource endpoints. { "message": "..." } — for mutations with no body.*

Default page size is 10 for public endpoints and 20 for admin endpoints. Maximum limit is capped at 100 (public) and 200 (admin logs) server-side to prevent excessive queries.

5.1 Authentication — /api/auth

Method	Endpoint	Description	Auth	SRS Ref.
POST	`/api/auth/register`	Register with .edu email; sends verification email	Public	FR-01, FR-02
GET	`/api/auth/verify-email/:token`	Verify email address (6-hour token TTL)	Public	FR-04
POST	`/api/auth/login`	Login → access token (15 min) + refresh token (7 days)	Public	FR-02
POST	`/api/auth/refresh`	Obtain new access token using refresh token	Public (refresh token)	FR-02
POST	`/api/auth/logout`	Log out; activity log entry created	Authenticated	FR-02
POST	`/api/auth/forgot-password`	Send password reset email (1-hour token TTL)	Public	FR-05
POST	`/api/auth/reset-password/:token`	Set new password using reset token	Public	FR-05

5.2 Post Management — /api/posts

Method	Endpoint	Description	Auth	SRS Ref.
GET	`/api/posts`	List active posts; supports ?domain, city, country,	Authenticated	FR-20

		stage, expertise, page, limit filters		
POST	`/api/posts`	Create a new post (owner = req.user)	Authenticated	FR-10
GET	`/api/posts/mine`	List all posts created by the authenticated user	Authenticated	FR-11
GET	`/api/posts/matches`	City-matched posts for the authenticated user's dashboard	Authenticated	FR-21
GET	`/api/posts/:id`	Post detail; NDA gate for meeting_only posts	Authenticated	FR-14
PUT	`/api/posts/:id`	Update post fields (owner only; locked if partner_found/expired)	Auth (owner)	FR-11
PATCH	`/api/posts/:id/status`	Status transition via STATUS_TRANSITIONS map	Auth (owner)	FR-12
DELETE	`/api/posts/:id`	Hard delete post + cascade (time_slots, NDA, meeting_requests). Note: hard-deleting removes audit trail — consider soft-delete (status = removed_by_user) for GDPR log integrity. Contrast with admin soft-delete (removed_by_admin).	Auth (owner) / Admin	FR-11

5.3 Meeting Requests — /api/meetings

Method	Endpoint	Description	Auth	SRS Ref.
POST	`/api/meetings`	Submit meeting request; NDA acceptance enforced for meeting_only posts	Authenticated	FR-30
GET	`/api/meetings`	List sent/received meetings; ?type=sent|received|all, ?status=...	Authenticated	FR-31
GET	`/api/meetings/:id`	Meeting detail with time slots; access restricted to parties	Auth (party)	FR-31
PATCH	`/api/meetings/:id/accept`	Post owner accepts request; post → meeting_scheduled	Auth (post owner)	FR-33

PATCH	`/api/meetings/:id/decline`	Post owner declines pending or accepted request	Auth (post owner)	FR-34
POST	`/api/meetings/:id/slots`	Requester proposes up to 5 time slots	Auth (requester)	FR-35
PATCH	`/api/meetings/:id/slots/:slotId`	Requester edits a proposed time slot	Auth (requester)	FR-35
DELETE	`/api/meetings/:id/slots/:slotId`	Requester deletes a proposed time slot	Auth (requester)	FR-35
PATCH	`/api/meetings/:id/slots/:slotId/confirm`	Other party confirms a slot; meeting → scheduled	Auth (non-proposer)	FR-35
DELETE	`/api/meetings/:id`	Cancel meeting request; post may revert to active	Auth (either party)	FR-36

5.3.1 Meeting Feedback — /api/meetings/:id/feedback (Planned)

> *No routes exist yet. The table schema is ready; backend implementation is deferred.*

Method	Endpoint	Description	Auth	SRS Ref.
POST	`/api/meetings/:id/feedback`	Submit trust + experience ratings after a scheduled meeting. One submission per user per meeting.	Auth (party)	SRS-FR-FBK-01
GET	`/api/meetings/:id/feedback`	Retrieve feedback for a meeting (visible to both parties only).	Auth (party)	SRS-FR-FBK-02

5.4 Admin — /api/admin

Method	Endpoint	Description	Auth	SRS Ref.
GET	`/api/admin/dashboard`	Stats (users, posts, pending meetings) + 10 recent activity logs	Admin	FR-40
GET	`/api/admin/users`	Paginated user list; ?role, ?status (active|suspended)	Admin	FR-40
GET	`/api/admin/users/:id`	User detail (excludes passwordHash, tokens)	Admin	FR-40
PATCH	`/api/admin/users/:id/suspend`	Toggle user account active/suspended state	Admin	FR-42
GET	`/api/admin/posts`	All posts with status/domain/date filters	Admin	FR-41

DELETE	`/api/admin/posts/:id`	Soft-delete post (status → removed_by_admin)	Admin	FR-41
GET	`/api/admin/logs`	Filtered activity log with pagination (?userId, actionType, from, to)	Admin	FR-43
GET	`/api/admin/logs/export`	Download filtered activity log as CSV	Admin	FR-44
DELETE	`/api/admin/logs/cleanup`	Delete logs older than N days (?olderThanDays=730)	Admin	FR-44
GET	`/api/admin/stats`	Compact stats JSON for charts	Admin	FR-40

5.5 User — /api/users

Method	Endpoint	Description	Auth	SRS Ref.
GET	`/api/users/profile`	Get authenticated user's profile	Authenticated	FR-06
PUT	`/api/users/profile`	Update profile fields (role and email cannot be changed)	Authenticated	FR-06
PUT	`/api/users/password`	Change password (requires current password verification)	Authenticated	FR-07
GET	`/api/users/export-data`	GDPR data export: profile + posts + meetings + NDAs as JSON download	Authenticated	NFR-GDPR-02
DELETE	`/api/users/account`	GDPR account deletion: anonymise all PII; posts expired	Authenticated	NFR-GDPR-01

6. UI Design

6.1 Screen List and Navigation

The frontend is a React 18 Single Page Application using React Router v6 for client-side navigation. Navigation is structured around three access levels: Public, Authenticated (engineer or healthcare), and Admin. The ThemeProvider component supports light/dark/system themes.

Screen	Description	Access Level
Landing Page	Public marketing page with platform description and registration call-to-action	Public
Registration / Login (AuthPage)	User sign-up with .edu email validation, role selection; login with JWT token handling	Public
Engineer Dashboard	Lists active posts, matched posts by city/expertise score, meeting notifications	Authenticated (engineer)
Healthcare Dashboard	Lists own posts with status, meeting request notifications, quick actions	Authenticated (healthcare)
Post Board (Board)	Filterable discovery board of all active posts with search, tags, and favourites	Public (view) / Auth (interact)
Post Detail (PostDetail)	Full post view, confidentiality-gated description, meeting request trigger	Authenticated
Create / Edit Post (PostForm)	Multi-field form for post creation and editing; draft/publish control	Authenticated
Meeting Request (MeetingRequest)	NDA acceptance gate + meeting request submission form	Authenticated
Meeting List (MeetingRequests)	Inbox/outbox view of all sent and received meeting requests	Authenticated
Meeting Detail (MeetingDetail)	Accept/decline, time slot proposal and confirmation, Google Calendar link	Authenticated (party)
Profile	View/edit profile, change password, GDPR data export, account deletion	Authenticated
MNDA Agreement	Full Mutual NDA legal text, standalone read-only page	Public
Admin Dashboard	Platform stats (users, posts, meetings), recent activity feed	Admin
Admin Users	Paginated user list, role/status filter, suspend/reactivate actions	Admin
Admin Posts	All posts with status/domain filters, soft-delete (removed_by_admin)	Admin
Admin Logs	Filtered activity log viewer, CSV export, log cleanup utility	Admin
Admin Stats	Aggregated platform statistics charts	Admin

FAQ / Troubleshooting / Contact	Help centre pages	Public

Navigation flow: Unauthenticated users land on the Landing Page and can navigate to the Board (read-only), FAQ, or Auth page. Successful login redirects to the role-specific dashboard (/dashboard/engineer or /dashboard/healthcare). The ProtectedRoute component enforces authentication and role restrictions for all guarded routes. Admin users are routed to the /admin subtree with its own AdminLayout component.

6.2 Frontend State Management

The application does not use a global state library (Redux, Zustand, or MobX). Authentication state is managed through a localStorage-based module (src/lib/auth.js) with a custom browser event bus:

- Auth storage: The access token, refresh token, and user object are stored in localStorage under the key healthai_auth as a JSON object. This is read on every page load by getAuth().
- Auth event bus: setAuth() and clearAuth() dispatch a custom DOM event (healthai-auth-changed) after updating localStorage. Components subscribe to this event via getAuthChangedEventName() to re-render on login/logout without a global store.
- Silent refresh: The current implementation does NOT include an automatic silent refresh mechanism in the frontend. The access token (15 min TTL) will expire and API calls will return HTTP 401. The user must manually log in again. A production-grade implementation should use an axios/fetch interceptor that catches 401 responses, attempts POST /api/auth/refresh with the stored refresh token, and retries the failed request.
- Component-level state: Each page component manages its own data-fetching state (loading, error, data) with React useState and useEffect hooks. There is no shared server-state cache (e.g. React Query / SWR).
- ProtectedRoute: Reads getAuth() synchronously on render. If no valid token is present, redirects to /auth. Role checking compares auth.user.role against the allowedRoles prop array.

6.3 Wireframes — Key Screens

The following descriptions correspond to the implemented UI screens:

Landing Page: Dark-themed hero section with animated gradient background, tagline, and two CTAs (Register / Discover). Navigation header with platform logo and theme toggle.

AuthPage: Single-page toggle between Login and Register forms. Register collects first name, last name, designation (role), and .edu email. Client-side .edu validation mirrors server validation. Animated panel transition between modes.

Dashboard (Engineer / Healthcare): Cards for Quick Actions, My Listings (own posts with status badges), Discover (matched posts by city/expertise with match score), and Meeting Notifications (inbox of pending/scheduled requests). Each card links to the relevant action page.

Board: A 3D card gallery of all active posts with live search (title/domain/expertise), tag and city filters, favourite toggling (localStorage), and toggle between gallery and list views. Unauthenticated users see posts but are prompted to log in on interaction.

Post Detail: Full post card with domain, stage, commitment level, location, and owner info. Description is gated behind NDA acceptance for meeting_only posts. CTA button triggers the meeting request flow.

Meeting Detail: Full lifecycle view: status badge, accept/decline buttons for owners, time slot proposal form for requesters, slot confirmation for the other party, confirmed meeting time with Google Calendar deep link.

Admin Dashboard: Stats widgets (total users, posts, pending meetings, active meetings) with sparkline charts. Recent activity feed showing the last 10 log entries with relative timestamps.

Platform Feedback Widget (FeedbackWidget): A floating button fixed to the bottom-left corner of every authenticated page (Layout.jsx). Opens an inline card with four emoji options (□ Terrible / □ Bad / □ Okay / □ Amazing), an optional email field, and a required message textarea. Auto-opens when the user navigates away from any /help/* page. Currently frontend-only — the onSubmit handler calls console.log. No backend endpoint or database persistence exists yet for platform feedback.

7. Security Design

The security architecture addresses all security and GDPR requirements from the SRS. The design follows a defence-in-depth approach with multiple independent layers of protection.

Security Requirement	Design Decision	SRS Ref.
Authentication	JWT stateless auth: access token (15 min) + refresh token (7 days). Bearer token required on all protected routes. Middleware: authenticate.js	NFR-SEC-01
Password Storage	bcrypt with saltRounds: 12. Password never stored in plain text or logs. Excluded from all Sequelize default scopes	NFR-SEC-02
Authorization (RBAC)	Three-role model: engineer, healthcare, admin. authorize.js middleware enforces role on each route. Post ownership checked inline in controllers	FR-03, NFR-SEC-03
Session Management	Access token TTL is 15 minutes. Silent refresh is NOT currently implemented in the frontend — when the token expires, API calls return HTTP 401 and the user must re-authenticate. A production implementation should add a fetch interceptor that calls /api/auth/refresh automatically on 401.	NFR-SEC-04
Refresh Token Storage & Revocation (Known Gap)	Current implementation is fully stateless — refresh tokens are signed JWTs stored client-side (localStorage) and are NOT persisted in the database. This means: (1) logout does not invalidate the refresh token server-side; (2) no token rotation is applied; (3) a stolen refresh token remains valid for 7 days. Mitigation path: store a refresh token hash in a db table (refresh_tokens) and delete on logout; implement token rotation on each /api/auth/refresh call. This is a known design trade-off accepted for the current phase.	NFR-SEC-01
Email Verification	.edu / .edu.tr email enforced via dual Joi custom validator. Regex: /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.edu(\.[a-z]{2})?$/i (international) and /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.edu\.tr$/i (Turkish). Blocked domain list: gmail.edu, yahoo.edu, hotmail.edu, outlook.edu. Verification token 6-hour TTL.	FR-01
Rate Limiting	express-rate-limit: general API 100 req/min (IP). Auth-specific limits defined in rateLimiter.js: login 5 req/15 min, register 3 req/hr. Window and max configurable via env vars.	NFR-SEC-05
HTTP Security Headers	Helmet.js: CSP (defaultSrc self, connectSrc allows blob/data for frontend assets), X-Content-Type-Options, X-Frame-Options. CORS origin restricted to FRONTEND_URL env var.	NFR-SEC-06

Input Validation	All request bodies pass through Joi schemas via validate() middleware before reaching controllers. abortEarly: false collects all errors; stripUnknown: true removes undeclared fields. Response: HTTP 400 + { message, errors[] }.	NFR-SEC-07
GDPR – Account Deletion	DELETE /api/users/account anonymises all PII in-place: email → deleted_<id>@removed.invalid, names → [Silindi], institution/city/country/expertise → null, passwordHash → [DELETED], isActive → false. User's posts set to expired.	NFR-GDPR-01
GDPR – Data Export	GET /api/users/export-data returns a JSON download bundle containing profile, posts, meetingRequests, and ndaAcceptances. Content-Disposition header triggers browser download.	NFR-GDPR-02
GDPR – IP Addresses	IP stored only as SHA-256 hex hash via crypto.createHash('sha256').update(ip).digest('hex') in logService.js. Raw IP is never written to any table.	NFR-GDPR-03
NDA Acceptance Audit	nda_acceptances table records user_id, post_id, accepted_at, ip_hash with UNIQUE(user_id, post_id) constraint. findOrCreate + update pattern ensures re-acceptance updates the audit timestamp.	FR-32
Request Body Limit	Express body parser limited to 10 KB (express.json({ limit: '10kb' })) to prevent payload-based DoS.	NFR-SEC-08
Hard Delete Audit Risk	User-initiated DELETE /api/posts/:id performs a hard delete with cascade (meeting_requests, nda_acceptances, time_slots removed). This destroys the audit trail for those records. Design recommendation: replace with a soft-delete status (removed_by_user), mirroring the admin's removed_by_admin approach, to preserve GDPR log integrity.	NFR-GDPR, FR-43
HTTPS	Mandatory in production deployment. Docker Compose does not terminate TLS; an upstream reverse proxy (nginx / Traefik / cloud LB) must handle HTTPS.	NFR-SEC-09

Logged action types in activity_logs: REGISTER, EMAIL_VERIFIED, LOGIN, FAILED_LOGIN, LOGOUT, PASSWORD_RESET, POST_CREATE, POST_CLOSE, POST_ADMIN_REMOVE, MEETING_REQUEST, MEETING_ACCEPT, MEETING_DECLINE, MEETING_CANCEL, SLOT_CONFIRMED, ACCOUNT_SUSPEND, ACCOUNT_REACTIVATE. Only critical events are logged to minimise data retention.

7.1 Error Handling Strategy

The application implements a two-layer error handling approach:

- Controller-level: Each controller wraps its logic in a try/catch block. Specific business rule violations return 4xx responses directly (e.g. 400 for invalid status transition, 403 for ownership failure, 404 for missing resource, 409 for duplicate registration). These are intentional, not unexpected errors.
- Global error handler: app.js registers a four-argument Express error handler as the last middleware. Any error propagated via next(err) or an unhandled throw reaches this handler, which returns HTTP 500. In production (NODE_ENV=production) the error message is replaced with a generic 'Sunucu hatası.' string to avoid leaking stack traces. In development the original err.message is returned.

HTTP status code conventions used across the API:

Status	Category	When Used
200	OK	Successful GET, PUT, PATCH, DELETE (with body)
201	Created	Successful POST that creates a new resource
400	Bad Request	Joi validation failure, invalid status transition, business rule violation
401	Unauthorized	Missing/invalid/expired JWT token
403	Forbidden	Authenticated but insufficient role or not the resource owner
404	Not Found	Resource does not exist or is not visible to the caller
409	Conflict	Duplicate registration email, already-active meeting request
500	Internal Server Error	Unexpected server-side error; message hidden in production

7.2 Migration Strategy

The current repository contains a migrations/sql/ directory with a single seed file (001_admin_enums.sql). Full Sequelize migration files have not yet been created — the system relies on Sequelize's DB_AUTO_SYNC / DB_SYNC_ALTER flags in development and Docker environments. This is explicitly unsuitable for production.

> *Production deployment requirement: DB_AUTO_SYNC must be set to false. Schema changes must be applied via versioned Sequelize migration files (sequelize db:migrate) before deploying new code.*

Recommended migration strategy:

- Create a migrations/ directory with numbered Sequelize migration files (e.g. 20260401000000-create-users.js) for each table defined in Section 4.2.
- Use sequelize-cli for migration execution: npx sequelize db:migrate (up) and npx sequelize db:migrate:undo (rollback).
- Add a pre-deploy migration step to the Docker entrypoint or CI/CD pipeline before the Express server starts.
- Never use sync({ force: true }) or sync({ alter: true }) in production — these risk data loss and schema drift.

8. State Diagrams

8.1 Post Lifecycle

> *States: DRAFT → ACTIVE → MEETING_SCHEDULED → PARTNER_FOUND / EXPIRED / REMOVED_BY_ADMIN*

The post lifecycle is enforced by the STATUS_TRANSITIONS map in postController.js. Only the transitions defined in this map are permitted. The expireStalePosts() function is called on every GET /api/posts request to enforce time-based expiry without requiring a background scheduler.

Transition	Trigger / Mechanism	SRS Ref.
Draft → Active	User clicks 'Publish' button on PostForm. Status validated via STATUS_TRANSITIONS map in postController.js	FR-12
Active → Meeting Scheduled	Post owner accepts a meeting request (PATCH /api/meetings/:id/accept). Post status updated automatically by meetingController	FR-30
Active → Draft	Post owner reverts post to draft (PATCH /api/posts/:id/status with status=draft)	FR-12
Meeting Scheduled → Partner Found	Post owner manually sets status to partner_found; triggers POST_CLOSE activity log entry	FR-12
Meeting Scheduled → Active	Meeting request is declined or cancelled; post reverts to active automatically	FR-36
Active / Meeting Scheduled → Expired	expiry_date < today detected on any GET /api/posts call (expireStalePosts()). Pending meeting requests auto-cancelled	FR-13
Any → Removed by Admin	Admin calls DELETE /api/admin/posts/:id (soft delete). Post hidden from public; only admin can view	FR-41

8.2 Meeting Request Lifecycle

> *States: → PENDING → ACCEPTED → SCHEDULED | PENDING / ACCEPTED → DECLINED | PENDING / ACCEPTED / SCHEDULED → CANCELLED*

The meeting request lifecycle is managed exclusively through meetingController.js. Each status change is accompanied by an email notification to the relevant party and an activity log entry. The lifecycle is tightly coupled to the parent post's status — accepting a meeting request automatically transitions the post to meeting_scheduled, and cancelling it may revert the post to active.

Transition	Trigger / Mechanism	SRS Ref.
→ Pending	Requester submits meeting request (POST /api/meetings). NDA accepted if post is meeting_only	FR-30

Pending → Accepted	Post owner calls PATCH /api/meetings/:id/accept. Email notification sent to requester	FR-33
Pending → Declined	Post owner calls PATCH /api/meetings/:id/decline. Email sent to requester	FR-34
Accepted → Scheduled	Requester proposes time slots; other party confirms one slot (PATCH .../slots/:slotId/confirm). confirmedSlot field set; both parties emailed	FR-35
Accepted → Declined	Post owner declines an already accepted request	FR-34
Pending/Accepted/Scheduled → Cancelled	Either party calls DELETE /api/meetings/:id. Post reverts to active if it was meeting_scheduled	FR-36
Any → Cancelled (auto)	Parent post expires (expiry_date < today); all pending requests for that post are auto-cancelled by expireStalePosts()	FR-13

9. SRS–SDD Traceability Matrix

The following table maps each SRS requirement to the corresponding SDD design element, ensuring all requirements are addressed in the design.

SRS Ref.	Requirement Summary	SDD Component	SDD Section
FR-01	.edu / .edu.tr email restriction at registration	AuthModule	Sec 3.1, 5.1, 7
FR-02	User registration and login	AuthModule	Sec 3.1, 5.1
FR-03	Role-based access control (RBAC)	AuthModule + Middleware	Sec 3.1, 7
FR-04	Email verification token flow	AuthModule, NotificationModule	Sec 3.1, 5.1
FR-05	Password reset via email token	AuthModule, NotificationModule	Sec 3.1, 5.1, 7
FR-10	Post creation with all fields	PostModule	Sec 3.1, 4.2.2, 5.2
FR-11	Post update (owner only)	PostModule	Sec 3.1, 5.2
FR-12	Post status lifecycle management	PostModule	Sec 4.2.2, 5.2, 8.1
FR-13	Post auto-expiry on expiry_date	PostModule (expireStalePosts)	Sec 5.2, 8.1
FR-14	Post confidentiality (meeting_only gate)	PostModule, MeetingModule	Sec 3.1, 4.2.2, 5.2
FR-20	Post discovery with filters (domain, city, stage, expertise)	PostModule + MatchingModule	Sec 3.1, 5.2
FR-21	City-based post matching for dashboard	MatchingModule	Sec 3.1, 5.2
FR-22	Match score calculation (expertise + city + stage + freshness)	MatchingModule (matchScore.js)	Sec 3.1
FR-30	Meeting request submission + NDA acceptance	MeetingModule	Sec 3.1, 4.2.3, 5.3
FR-31	Duplicate meeting request prevention	MeetingModule	Sec 5.3
FR-32	NDA acceptance audit trail	MeetingModule + AuditModule	Sec 4.2.5, 7
FR-33	Meeting request accept flow	MeetingModule	Sec 5.3, 8.2
FR-34	Meeting request decline flow	MeetingModule	Sec 5.3, 8.2
FR-35	Time slot proposal and confirmation	MeetingModule	Sec 4.2.4, 5.3, 8.2

FR-36	Meeting cancellation by either party	MeetingModule	Sec 5.3, 8.2
FR-40	Admin user listing and filtering	AdminModule	Sec 3.1, 5.4
FR-41	Admin post soft-delete	AdminModule	Sec 3.1, 5.4
FR-42	Admin account suspend/reactivate	AdminModule	Sec 3.1, 5.4
FR-43	Activity log viewer with filters	AuditModule	Sec 3.1, 5.4
FR-44	Activity log CSV export	AuditModule	Sec 5.4
NFR-GDPR-01	GDPR account deletion (data anonymisation)	UserModule	Sec 5.5, 7
NFR-GDPR-02	GDPR personal data export (JSON bundle)	UserModule	Sec 5.5, 7
NFR-GDPR-03	IP address hashing (SHA-256)	AuditModule	Sec 4.2.6, 7
NFR-SEC-01	JWT access + refresh token auth	AuthModule	Sec 3.1, 7
NFR-SEC-02	bcrypt password hashing (saltRounds 12)	AuthModule	Sec 7
NFR-SEC-05	Rate limiting (express-rate-limit)	Middleware (rateLimiter.js)	Sec 7
NFR-SEC-06	HTTP security headers (Helmet.js + CORS)	app.js	Sec 7
NFR-SEC-07	Input validation (Joi schemas)	Middleware (validate.js)	Sec 7
NFR-DEP-01	Containerised deployment (Docker + Compose)	Infrastructure	Sec 2.3
SRS-FR-FBK-01	Post-meeting feedback submission (trust + experience ratings)	MeetingFeedbackModule (Planned)	Sec 3, 4.2.7, 5.3.1
SRS-FR-FBK-02	Retrieve feedback for a scheduled meeting	MeetingFeedbackModule (Planned)	Sec 5.3.1

10. Appendices

10.1 Project File Structure

The repository follows a monorepo layout with frontend sources at the root level and backend sources in /backend:

```
health-ai-cocreation/
├── src/                          # React frontend (Vite)
│   ├── pages/                    # Route-level page components
│   ├── components/               # Shared UI components
│   ├── lib/                      # auth.js, matchScore.js, googleCalendar.js
│   └── App.jsx                   # Router configuration
├── public/                       # Static assets (mesh images)
├── backend/
│   └── src/
│       ├── config/               # database.js, env.js, corsOptions.js
│       ├── models/               # Sequelize models + associations (index.js)
│       ├── controllers/          # authController, postController, meetingController,
│       │                         # userController, adminController
│       ├── routes/               # auth.js, posts.js, meetings.js, users.js, admin.js
│       ├── middleware/           # authenticate.js, authorize.js, rateLimiter.js,
validate.js
│       ├── services/             # emailService.js, logService.js
│       └── validations/          # Joi schemas: authSchemas, postSchemas,
meetingSchemas
├── docker-compose.yml
├── Dockerfile
└── package.json                  # Root scripts: dev:full, start:full
```

10.2 Environment Variables Reference

All sensitive configuration is managed through environment variables. The following variables are required:

Variable	Default / Example	Description
`PORT`	3001	Express server port
`NODE_ENV`	development | production	Runtime environment
`DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USER` / `DB_PASS`	—	MySQL connection credentials
`JWT_ACCESS_SECRET` / `JWT_REFRESH_SECRET`	—	JWT signing secrets (min 32 chars recommended)
`JWT_ACCESS_EXPIRY` / `JWT_REFRESH_EXPIRY`	15m / 7d	Token TTL values
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS`	—	SMTP server for Nodemailer
`EMAIL_FROM`	—	Sender name and address
`FRONTEND_URL`	http://localhost:5173	CORS whitelist + email link base

BCRYPT_SALT_ROUNDS	12	bcrypt cost factor
DB_AUTO_SYNC / DB_SYNC_ALTER	true / false	Sequelize auto-sync options (dev only)
SKIP_EMAIL_VERIFICATION	true (dev) / false (prod)	Bypass email verification in development
DB_CONNECT_RETRIES / DB_CONNECT_DELAY_MS	20 / 3000	DB connection retry config (Docker)

10.3 Match Score Algorithm

The client-side match score algorithm in src/lib/matchScore.js calculates a compatibility percentage between the authenticated user's profile and a post. The weighted scoring model is:

- Expertise match (+50 pts): Token-based fuzzy text match between user.expertise and post.required_expertise. Normalised (NFKD, lowercase), then checked for substring or token overlap.
- City match (+30 pts): Exact equality after normalisation between user.city and post.city.
- Prototype stage (+10 pts): Bonus if post.project_stage === 'prototype' (most actionable phase).
- Freshness (+10 pts): Bonus if post was created within the last 3 days.

Maximum score is 100. The score is displayed as a percentage badge on dashboard discovery cards to guide users towards the most relevant collaboration opportunities.